TRABAJO PRACTICO 4

MANIPULACION DE GRAFICOS 2D Y 3D CON PROCESSING

INTRO al desarrollo en motores de VideoJuegos

Objetivos

- ✓ Diseñar aplicando POO.
- ✓ Manipular algorítmicamente representaciones geométricas en 2D y 3D.
- ✓ Resolver diferentes problemas que requieran la aplicación de conceptos de geometría algorítmica para implementar programas usando animación de objetos e interacción con el usuario mediante dispositivos de E/S.
- ✓ Implementar usando Processing, Unity, Godot, UDK u otros entornos de desarrollo a elección del alumno.

Bibliografía

- ✓ La recomendada en el programa de estudio y en el material didáctico provisto.

Procedimiento general

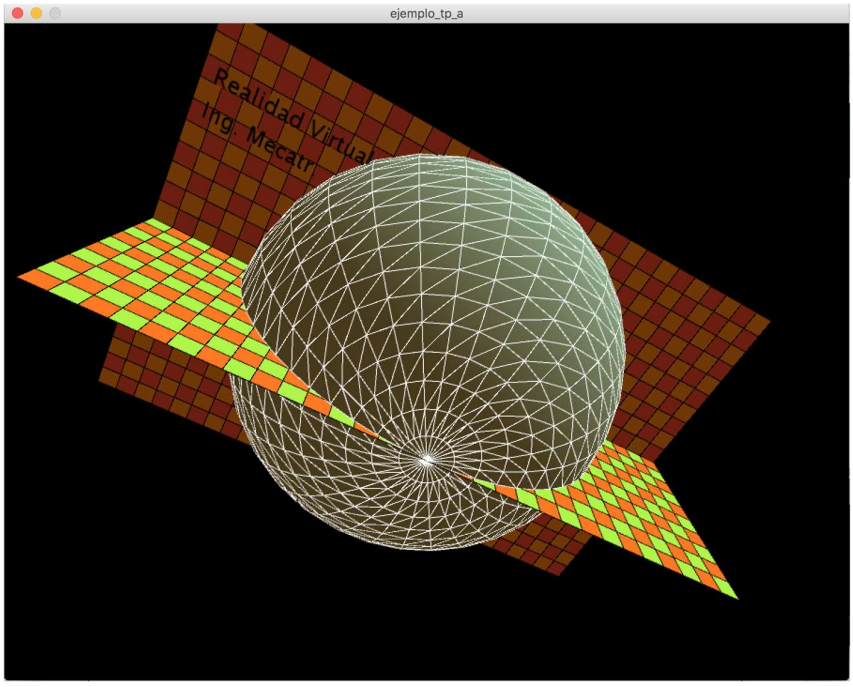
- ✓ Diseñar las clases que permitan tanto controlar la interacción con el usuario como realizar el despliegue gráfico solicitado.
- ✓ Aplicar criterios de diseño y programación OO.

Herramientas

- ✓ Software de programación: IDEs varios, Processing, OpenGL, X3D[VRML], motores gráficos varios.
- ✓ Descripciones y ejemplos de códigos fuente disponibles en los enlaces web indicados en los apuntes.
- ✓ Códigos fuente de ejemplo provistos en clase.

Consignas de programación específicas

1. Implementar, en Processing, un pequeño ambiente 3D donde se presenten 2 o más planos capaces de rotar sobre alguno de sus ejes, como se muestra en la siguiente imagen a modo de ejemplo, pero se debe proponer una figura original:



El usuario debe poder aplicar (o cambiar) un sentido de rotación al conjunto de planos según 1 ejes de rotación (vertical u horizontal) al pulsar cualquiera de las 4 teclas de flecha, detener (al pulsar D) o bien reiniciar la animación (al pulsar R). Cuando se acciona ésta última, debe producirse un cambio en el diseño de cada superficie (ya sea mediante uso de imágenes diferentes o relleno con patrones).

Asuma un tipo de iluminación a su criterio, de modo de lograr un buen efecto/resalte.

2. Realizar una prueba de cada una de las herramientas de desarrollo de videojuegos presentada en clase a modo de "hola mundo" de la misma, algo básico que permita demostrar el desarrollo de la misma, mostrando una figura en pantalla generando movimiento de rotación sobre su eje horizontal y vertical mediante las teclas del cursor de teclado izq-der, arr-ab.

3. Elegir una de las herramientas de desarrollo investigadas e Implementar el mismo ambiente gráfico del punto **1**.

4. Desarrollar el mismo ejercicio práctico del parcial en la herramienta elegida.

Ejercicio 1

El código se armó separando la lógica en dos partes principales: una parte controla la escena completa y otra representa cada plano de forma individual.

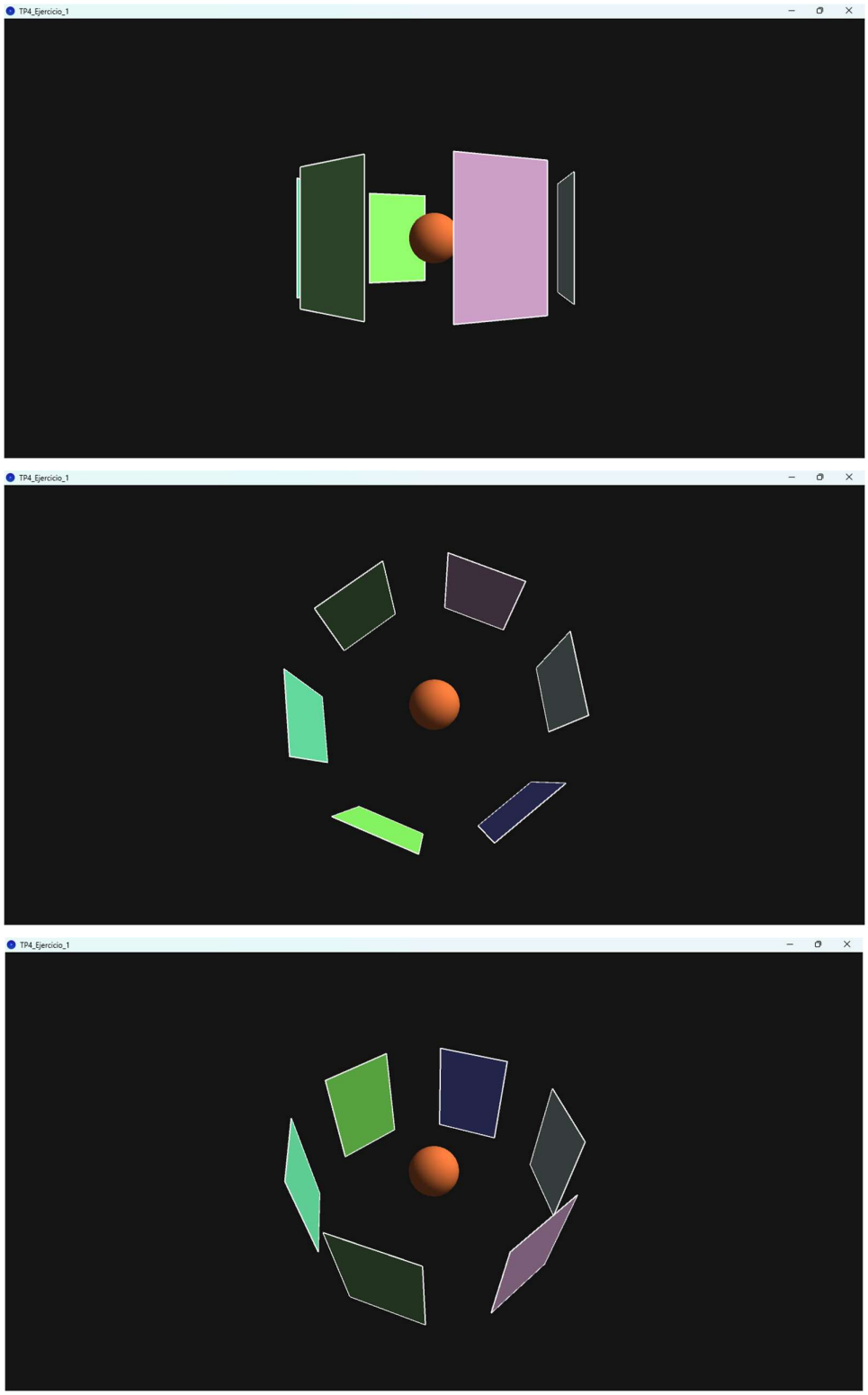
Primero se crea un arreglo de objetos `Plano3D`. Cada objeto guarda su posición, orientación, tamaño y color. Los planos se distribuyen alrededor de un círculo usando `sin()` y `cos()`, así no quedan todos amontonados como muebles en una mudanza.

En `draw()` se dibuja continuamente la escena. Ahí se aplica la iluminación, se traslada el origen al centro de la pantalla y se usan `rotateX()` y `rotateY()` para hacer girar todo el conjunto de planos.

La interacción con el usuario se maneja con `keyPressed()`. Las flechas cambian el eje y el sentido de giro, la tecla `D` detiene la animación y la tecla `R` reinicia la posición y cambia el diseño de los planos.

Para no mezclar las transformaciones de un objeto con otro se usan `pushMatrix()` y `popMatrix()`. Cada plano se mueve a su posición con `translate()`, se orienta con `rotateY()` y finalmente se dibuja usando cuatro vértices con `beginShape(QUADS)`.

En resumen, se aplicó POO porque cada plano es un objeto independiente, pero todos son controlados como un conjunto desde el programa principal. Así el código queda más ordenado, reutilizable y fácil de modificar.


TP4_Ejercicio_1
TP4_Ejercicio_1
TP4_Ejercicio_1